

LAPORAN PROYEK
MATA KULIAH KECERDASAN BUATAN

IMPLEMENTASI ALGORITMA A* (A-STAR) DAN DIJKSTRA
BERBASIS OSRM PADA SISTEM NAVIGASI RUTE KAMPUS
UNIVERSITAS BENGKULU (UNIB Route Navigation System)



Disusun Oleh :

R. Noor Fikhri

G1A024075

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU

2026

DAFTAR ISI

DAFTAR ISI.....	i
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	2
1.4 Manfaat Penelitian	2
1.5 Batasan Masalah.....	3
BAB II LANDASAN TEORI	4
2.1 Sistem Informasi Geografis Berbasis Web (Web GIS)	4
2.2 Algoritma Dijkstra	4
2.3 Algoritma A* (A-Star)	5
2.4 Perbandingan Dijkstra dan A*	5
2.5 Open Source Routing Machine (OSRM).....	6
2.6 Leaflet.js	6
BAB III METODOLOGI DAN IMPLEMENTASI.....	7
3.1 Metodologi Pengembangan Sistem	7
3.2 Arsitektur Sistem.....	7
3.3 Implementasi Kode Sumber	7
3.3.1 Struktur Dokumen HTML (index.html)	7
3.3.2 Struktur Titik Lokasi	11
3.3.3 Sistem Zoom Tiering Dinamis	13
3.3.4 Fungsi Pencarian Rute Terpendek.....	15
BAB IV HASIL DAN PEMBAHASAN.....	18
4.1 Tampilan Antarmuka Utama Aplikasi.....	18
4.2 Pengujian Fitur Zoom Tiering	19
4.2.1 Level Zoom Tier 1.....	19
4.2.2 Level Zoom Tier 2.....	20
4.2.3 Level Zoom Tier 3.....	21
4.2.4 Level Zoom Tier 4.....	21
4.3 Pengujian Hasil Pencarian Rute Terpendek.....	22

4.3.1 Rute Mode Kendaraan	22
4.3.2 Rute Mode Jalan Kaki.....	23
BAB V KESIMPULAN DAN SARAN	25
5.1 Kesimpulan	25
5.2 Saran.....	25
DAFTAR PUSTAKA.....	26

BAB I

PENDAHULUAN

1.1 Latar Belakang

Universitas Bengkulu (UNIB) merupakan perguruan tinggi negeri yang memiliki kawasan kampus dengan luas yang signifikan dan terdiri dari berbagai gedung, fakultas, fasilitas umum, serta infrastruktur pendukung akademik yang tersebar di seluruh area kampus. Kompleksitas tata letak kampus tersebut kerap menjadi kendala nyata bagi mahasiswa baru, tamu institusi, dan bahkan sivitas akademika dalam menavigasi rute menuju lokasi yang dituju secara efisien.

Di era revolusi industri 4.0 dan pesatnya perkembangan teknologi informasi geospasial, solusi navigasi berbasis web telah menjadi kebutuhan yang krusial. Sistem Informasi Geografis (SIG) berbasis web, atau yang lazim disebut *Web GIS*, menawarkan kemampuan untuk menyajikan data spasial secara interaktif langsung melalui peramban (*browser*), tanpa memerlukan instalasi perangkat lunak khusus di sisi pengguna. Kombinasi teknologi ini dengan algoritma kecerdasan buatan untuk pencarian jalur terpendek membuka peluang pengembangan sistem navigasi yang akurat, efisien, dan mudah diakses.

Dalam konteks kecerdasan buatan, permasalahan pencarian rute terpendek (*shortest path problem*) adalah salah satu domain yang paling banyak dikaji. Dua algoritma yang paling relevan dan banyak diimplementasikan dalam literatur akademik adalah Algoritma Dijkstra dan Algoritma A* (A-Star). Algoritma Dijkstra, yang pertama kali dirumuskan oleh Edsger W. Dijkstra pada tahun 1959, bekerja dengan menelusuri seluruh simpul graf secara sistematis dan menjamin penemuan jalur dengan bobot minimum. Sementara itu, Algoritma A* merupakan pengembangan heuristik dari Dijkstra yang menggunakan fungsi estimasi jarak untuk mengarahkan proses pencarian, sehingga secara teori lebih efisien pada ruang pencarian yang besar.

Sistem navigasi yang dikembangkan dalam laporan ini, bernama UNIB Route Navigation System, merupakan sebuah aplikasi *Web GIS* yang dibangun menggunakan teknologi *front-end* modern, yakni HTML5, CSS3 dengan paradigma desain *glassmorphism*, dan JavaScript. Sistem ini memanfaatkan pustaka pemetaan Leaflet.js dan mengintegrasikan mesin perutean Open Source

Routing Machine (OSRM) sebagai *backend* kalkulasi rute, yang secara internal mengimplementasikan optimasi berbasis Multilevel Dijkstra dan Contraction Hierarchies sebuah derivat canggih dari Algoritma Dijkstra konvensional yang digabungkan dengan prinsip heuristik A*. Sistem juga dilengkapi dengan fitur inovatif berupa *Zoom Tiering* dinamis untuk manajemen visibilitas penanda peta, serta kemampuan *parsing* data spasial dari format KML (*Keyhole Markup Language*).

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah yang diangkat dalam laporan ini adalah sebagai berikut:

1. Bagaimana merancang dan mengimplementasikan sistem navigasi rute kampus UNIB berbasis *Web GIS* yang mampu memanfaatkan algoritma kecerdasan buatan untuk menentukan jalur terpendek?
2. Bagaimana mengintegrasikan mesin perutean OSRM yang secara internal menggunakan derivat Dijkstra dan A* ke dalam antarmuka berbasis Leaflet.js?
3. Bagaimana merancang mekanisme *Zoom Tiering* dinamis guna meningkatkan keterbacaan peta dan pengalaman pengguna secara keseluruhan?

1.3 Tujuan Penelitian

Tujuan dari pengembangan sistem ini adalah sebagai berikut:

1. Merancang dan membangun sistem navigasi rute kampus UNIB berbasis *Web GIS* yang fungsional dan responsif.
2. Mengimplementasikan integrasi Algoritma A* dan Dijkstra melalui OSRM API untuk kalkulasi jalur terpendek antar titik lokasi di dalam kampus.
3. Mengembangkan fitur *Zoom Tiering* dinamis yang secara otomatis mengatur visibilitas penanda peta sesuai dengan tingkat *zoom* yang aktif. Keempat, mengintegrasikan data spasial eksternal dari format file KML ke dalam sistem peta interaktif.

1.4 Manfaat Penelitian

Manfaat dari pengembangan sistem ini adalah sebagai berikut:

1. Dari sisi akademis, laporan ini memberikan kontribusi terhadap pemahaman teoritis dan praktis mengenai implementasi algoritma pencarian jalur terpendek khususnya Dijkstra dan A* dalam konteks aplikasi *Web GIS* nyata di lingkungan kampus.
2. Dari sisi praktis, sistem yang dibangun dapat dimanfaatkan secara langsung oleh mahasiswa baru, tamu, dan seluruh sivitas akademika Universitas Bengkulu sebagai alat bantu navigasi yang efisien.
3. Bagi pengembang, proyek ini menjadi referensi nyata mengenai cara mengintegrasikan berbagai teknologi *front-end* HTML, CSS, JavaScript, Leaflet.js dengan layanan API perutean eksternal berbasis algoritma AI.

1.5 Batasan Masalah

Agar pembahasan tetap terfokus dan terukur, terdapat beberapa batasan yang diterapkan dalam pengembangan sistem ini. Sistem beroperasi sepenuhnya berbasis *front-end* tanpa *server-side* khusus, sehingga kalkulasi rute bergantung pada konektivitas jaringan internet menuju server OSRM publik. Data titik lokasi yang digunakan mencakup 19 titik utama yang didefinisikan secara manual di dalam variabel `manualPoints`, serta titik-titik tambahan yang bersumber dari file `data_kampus.kml`. Sistem hanya mendukung dua mode transportasi, yaitu kendaraan (*driving*) dan jalan kaki (*walking*). Peta dasar (*base map*) menggunakan *tile* dari server OpenStreetMap Jerman.

BAB II

LANDASAN TEORI

2.1 Sistem Informasi Geografis Berbasis Web (Web GIS)

Sistem Informasi Geografis (SIG) adalah suatu sistem komputerisasi yang dirancang untuk menangkap, menyimpan, memanipulasi, menganalisis, mengelola, dan menyajikan data yang terikat secara spasial (georeferenced). *Web GIS* merupakan evolusi dari SIG konvensional yang memanfaatkan infrastruktur internet sebagai medium distribusi data dan antarmuka pengguna. Dalam paradigma ini, seluruh pemrosesan dan visualisasi peta dapat dilakukan langsung di dalam peramban web pengguna (*client-side*), menjadikan sistem lebih aksesibel tanpa memerlukan instalasi aplikasi khusus. Aplikasi UNIB Route Navigation System yang dibahas dalam laporan ini merupakan implementasi nyata dari konsep *Web GIS* berbasis *client-side* murni.

2.2 Algoritma Dijkstra

Algoritma Dijkstra adalah algoritma pencarian jalur terpendek (*shortest path algorithm*) pada sebuah graf berbobot yang dikembangkan oleh ilmuwan komputer Belanda, Edsger Wybe Dijkstra, dan dipublikasikan pada tahun 1959. Algoritma ini bekerja secara *greedy* dengan prinsip selalu memilih simpul yang memiliki jarak kumulatif terkecil dari simpul asal (*source vertex*) pada setiap iterasinya.

Secara formal, misalkan terdapat sebuah graf $G = (V, E)$ dengan V adalah himpunan simpul (*vertex*) dan E adalah himpunan sisi (*edge*) berbobot non-negatif. Algoritma Dijkstra menjaga sebuah struktur data, biasanya berupa *min-priority queue*, yang menyimpan estimasi jarak terpendek sementara $d(v)$ untuk setiap simpul v . Proses algoritmanya adalah sebagai berikut.

Pertama, inisialisasi: $d(s) = 0$ untuk simpul asal s , dan $d(v) = \infty$ untuk semua simpul lainnya. Kedua, pada setiap iterasi, pilih simpul u dengan nilai $d(u)$ minimum dari antrian yang belum diproses. Ketiga, untuk setiap tetangga v dari u , lakukan operasi *relaxation*: jika $d(u) + w(u,v) < d(v)$, perbarui $d(v) = d(u) + w(u,v)$. Keempat, ulangi hingga semua simpul terpilih.

Kompleksitas waktu algoritma Dijkstra adalah $O(|E| + |V| \log |V|)$ dengan implementasi menggunakan *Fibonacci heap*, atau $O(|V|^2 + |E|)$ dengan

implementasi sederhana menggunakan matriks. Sifat utama algoritma ini adalah optimalitas yang terjamin: ia selalu menemukan jalur terpendek yang sesungguhnya selama bobot sisi bersifat non-negatif. Namun, kelemahannya terletak pada sifatnya yang menelusuri seluruh arah (*non-directional*) tanpa panduan, sehingga kurang efisien untuk graf yang sangat besar.

2.3 Algoritma A* (A-Star)

Algoritma A* (A-Star) merupakan algoritma pencarian jalur terpendek yang merupakan generalisasi dan pengembangan heuristik dari Algoritma Dijkstra. A* pertama kali diperkenalkan oleh Peter Hart, Nils Nilsson, dan Bertram Raphael pada tahun 1968. Keunggulan utama A* terletak pada penggunaan fungsi evaluasi $f(n)$ yang memandu proses pencarian secara terarah menuju simpul tujuan, sehingga secara signifikan dapat mengurangi jumlah simpul yang perlu dieksplorasi. Fungsi evaluasi A* didefinisikan sebagai:

$$f(n) = g(n) + h(n)$$

Di mana $g(n)$ adalah biaya aktual (jarak sesungguhnya) dari simpul awal ke simpul n saat ini, dan $h(n)$ adalah fungsi heuristik yang merupakan estimasi biaya dari simpul n ke simpul tujuan. Fungsi heuristik yang paling umum digunakan dalam konteks navigasi geografis adalah *Euclidean distance* (jarak garis lurus) atau *Haversine distance* untuk koordinat berbasis bola bumi. Syarat agar A* dapat menjamin optimalitas adalah fungsi heuristik $h(n)$ harus bersifat admissible, artinya tidak boleh melebihi-lebihkan biaya sebenarnya menuju tujuan ($h(n) \leq h^*(n)$).

Perbedaan mendasar antara A* dan Dijkstra adalah bahwa Dijkstra memperlakukan semua arah ekspansi secara merata ($h(n) = 0$), sedangkan A* menggunakan informasi heuristik untuk memprioritaskan simpul-simpul yang secara geometris lebih "menjanjikan" dalam mengarah ke tujuan. Hal ini menjadikan A* jauh lebih efisien dalam skenario navigasi peta nyata dengan ruang pencarian yang besar.

2.4 Perbandingan Dijkstra dan A*

Secara komparatif, kedua algoritma ini memiliki karakteristik yang saling melengkapi. Dijkstra unggul dalam kasus pencarian jalur dari satu sumber ke *semua* simpul lainnya, sementara A* lebih efisien untuk pencarian jalur dari satu

titik asal ke satu titik tujuan yang spesifik. Dari sisi hasil, keduanya menjamin jalur optimal selama persyaratan masing-masing terpenuhi. Namun dari sisi efisiensi komputasi, A* umumnya mengeksplorasi lebih sedikit simpul berkat panduan heuristiknya, sehingga lebih cepat dalam skenario navigasi nyata.

2.5 Open Source Routing Machine (OSRM)

Open Source Routing Machine (OSRM) adalah mesin perutean (*routing engine*) berkinerja tinggi yang bersifat *open-source*, ditulis dalam bahasa C++, dan dirancang untuk bekerja dengan data jaringan jalan dari OpenStreetMap. OSRM secara signifikan berbeda dari implementasi naif Dijkstra karena menggunakan teknik Contraction Hierarchies (CH) dan Multilevel Dijkstra sebuah metode pra-pemrosesan graf yang menyederhanakan jaringan jalan dengan membangun hierarki simpul berdasarkan "kepentingan" topologisnya, sehingga waktu kueri dapat direduksi hingga di bawah 1 milidetik bahkan untuk dataset sekelas benua Eropa.

Dalam sistem yang dibangun, OSRM diakses melalui library Leaflet Routing Machine yang berkomunikasi dengan server OSRM publik menggunakan protokol OSRM v1 API. OSRM mendukung berbagai profil transportasi, termasuk *driving* (kendaraan bermotor) dan *foot* (jalan kaki), yang masing-masing memiliki parameter kecepatan dan batasan akses jalan yang berbeda.

2.6 Leaflet.js

Leaflet.js adalah pustaka JavaScript *open-source* yang ringan dan populer untuk pembuatan peta web interaktif. Leaflet menyediakan API yang kaya untuk menambahkan lapisan (*layer*) peta, penanda (*marker*), bentuk geometri, kontrol UI, dan penanganan *event* peta. Dalam konteks sistem ini, Leaflet berperan sebagai *rendering engine* utama yang menampilkan *tile* peta OpenStreetMap, mengelola penanda POI (*Point of Interest*), dan memvisualisasikan garis rute hasil kalkulasi OSRM.

BAB III

METODOLOGI DAN IMPLEMENTASI

3.1 Metodologi Pengembangan Sistem

Pengembangan UNIB Route Navigation System mengikuti metodologi *Rapid Application Development* (RAD), yang terdiri atas tahapan perencanaan kebutuhan, desain sistem dan antarmuka, implementasi kode, serta pengujian fungsional. Metodologi ini dipilih karena cocok untuk proyek *front-end* berbasis prototipe yang iteratif, di mana perubahan desain dan fungsionalitas dapat dilakukan dengan cepat tanpa siklus pengembangan yang panjang.

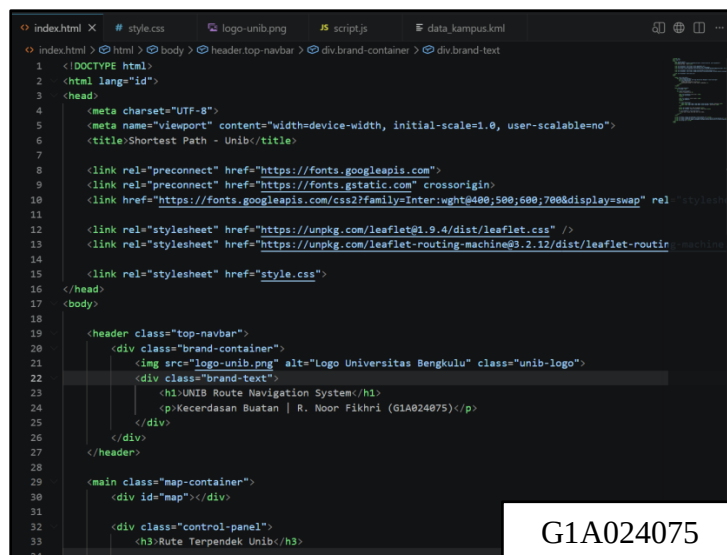
3.2 Arsitektur Sistem

Sistem dibangun di atas arsitektur *client-side* penuh (*full client-side architecture*). Seluruh logika aplikasi, termasuk manajemen data, rendering peta, dan penanganan *event*, berjalan di peramban pengguna. Komunikasi dengan server eksternal hanya terjadi dalam dua skenario: pertama, saat mengunduh *tile* peta dari server OpenStreetMap; dan kedua, saat mengirimkan permintaan rute ke server OSRM publik dan menerima respons berupa data koordinat jalur dalam format JSON.

3.3 Implementasi Kode Sumber

Berikut adalah analisis mendalam terhadap setiap segmen kode kritis yang membentuk sistem UNIB Route Navigation System.

3.3.1 Struktur Dokumen HTML (index.html)



```
1 <!DOCTYPE html>
2 <html lang="id">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=no">
6   <title>Shortest Path - Unib</title>
7
8   <link rel="preconnect" href="https://fonts.googleapis.com">
9   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
10  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap" rel="stylesheet">
11
12  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
13  <link rel="stylesheet" href="https://unpkg.com/leaflet-routing-machine@3.2.12/dist/leaflet-routing-machine.css" />
14
15  <link rel="stylesheet" href="style.css">
16 </head>
17 <body>
18
19   <header class="top-navbar">
20     <div class="brand-container">
21       
22       <div class="brand-text">
23         <h1>UNIB Route Navigation System</h1>
24         <p>Kecerdasan Buatan | R. Noor Fikhri (G1A024075)</p>
25       </div>
26     </div>
27   </header>
28
29   <main class="map-container">
30     <div id="map"></div>
31
32     <div class="control-panel">
33       <h3>Rute Terpendek Unib</h3>
34     </div>
35   </main>
36 </body>
```

Gambar 3.1 Source Code index.html

Source Code:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0,
user-scalable=no">
  <title>Shortest Path - Unib</title>

  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;70
0&display=swap" rel="stylesheet">

  <link                                rel="stylesheet"
href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
  <link      rel="stylesheet"      href="https://unpkg.com/leaflet-routing-
machine@3.2.12/dist/leaflet-routing-machine.css" />

  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header class="top-navbar">
    <div class="brand-container">
      
      <div class="brand-text">
        <h1>UNIB Route Navigation System</h1>
        <p>Kecerdasan Buatan | R. Noor Fikhri (G1A024075)</p>
      </div>
```

```

        </div>
    </header>

    <main class="map-container">
        <div id="map"></div>

        <div class="control-panel">
            <h3>Rute Terpendek Unib</h3>

            <label for="startPoint">Titik Asal:</label>
            <select id="startPoint">
            </select>

            <label for="endPoint">Titik Tujuan:</label>
            <select id="endPoint">
            </select>

            <div class="mode-group">
                <label><input type="radio" name="mode" value="driving"
checked> Kendaraan</label>
                <label><input type="radio" name="mode" value="walking"> Jalan
Kaki</label>
            </div>

            <button id="routeBtn">Cari Rute Terpendek</button>
            <button id="resetBtn" class="refresh-btn"> Reset Peta</button>

            <div id="infoResult" class="info-result">
                Pilih asal, tujuan, dan mode lalu klik tombol.
            </div>
        </div>
    </main>

```

```

<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
<script                                src="https://unpkg.com/leaflet-routing-
machine@3.2.12/dist/leaflet-routing-machine.js"></script>
<script      src="https://api.tiles.mapbox.com/mapbox.js/plugins/leaflet-
omnivore/v0.3.1/leaflet-omnivore.min.js"></script>
<script src="script.js"></script>
</body>
</html>

```

Penjelasan Source Code:

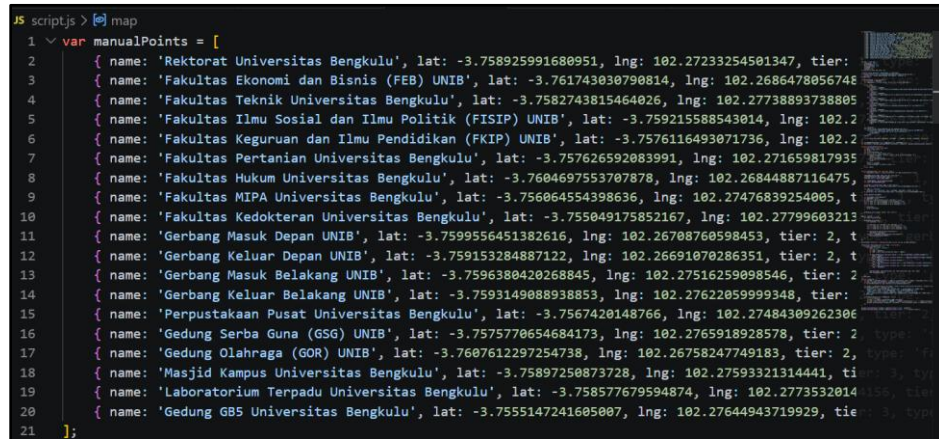
Segmen ini mendefinisikan kerangka struktural (*scaffold*) dokumen HTML yang berfungsi sebagai fondasi dan titik masuk (*entry point*) dari seluruh aplikasi, sekaligus mendeklarasikan semua dependensi pustaka eksternal yang dibutuhkan sistem.

Tag `<!DOCTYPE html>` dan `<html lang="id">` mendeklarasikan dokumen sebagai HTML5 dengan bahasa antarmuka Bahasa Indonesia. Elemen `<meta name="viewport">` dengan atribut `user-scalable=no` memastikan tampilan responsif pada perangkat *mobile* dengan mencegah *zoom* manual oleh pengguna, menjaga konsistensi tampilan peta. Di dalam `<head>`, terdapat tiga *stylesheet* yang dimuat secara berurutan: CSS dari Leaflet versi 1.9.4 untuk gaya dasar peta, CSS dari Leaflet Routing Machine versi 3.2.12 untuk antarmuka perutean, dan terakhir `style.css` sebagai *stylesheet* kustom yang menimpa (*override*) gaya-gaya bawaan.

Bagian `<body>` disusun menjadi dua blok struktural utama: elemen `<header>` dengan kelas `top-navbar` yang menampung branding aplikasi, dan elemen `<main>` dengan kelas `map-container` yang menampung dua komponen inti, yaitu `<div id="map">` sebagai wadah rendering peta Leaflet, dan `<div class="control-panel">` sebagai panel kontrol navigasi. Di dalam panel kontrol terdapat elemen `<select id="startPoint">` dan `<select id="endPoint">` yang akan diisi secara dinamis oleh JavaScript sebagai *dropdown* pemilihan lokasi. Elemen `<div id="infoResult">` berfungsi sebagai wadah output hasil komputasi rute. Di akhir `<body>`, pustaka JavaScript dimuat secara berurutan

leaflet.js, leaflet-routing-machine.js, leaflet-omnivore.min.js diikuti oleh script.js milik aplikasi. Urutan pemuatan ini bersifat kritis karena script.js bergantung pada seluruh pustaka yang telah dimuat sebelumnya.

3.3.2 Struktur Titik Lokasi



```

1  var manualPoints = [
2    { name: 'Rektorat Universitas Bengkulu', lat: -3.758925991680951, lng: 102.27233254501347, tier:
3    { name: 'Fakultas Ekonomi dan Bisnis (FEB) UNIB', lat: -3.761743030790814, lng: 102.2686478056748
4    { name: 'Fakultas Teknik Universitas Bengkulu', lat: -3.7582743815464026, lng: 102.27738893738805
5    { name: 'Fakultas Ilmu Sosial dan Ilmu Politik (FISIP) UNIB', lat: -3.759215588543014, lng: 102.2
6    { name: 'Fakultas Keguruan dan Ilmu Pendidikan (FKIP) UNIB', lat: -3.7576116493071736, lng: 102.2
7    { name: 'Fakultas Pertanian Universitas Bengkulu', lat: -3.757626592083991, lng: 102.271659817935
8    { name: 'Fakultas Hukum Universitas Bengkulu', lat: -3.7604697553707878, lng: 102.26844887116475,
9    { name: 'Fakultas MIPA Universitas Bengkulu', lat: -3.756064554398636, lng: 102.27476839254005, t
10   { name: 'Fakultas Kedokteran Universitas Bengkulu', lat: -3.755049175852167, lng: 102.27799603213
11   { name: 'Gerbang Masuk Depan UNIB', lat: -3.7599556451382616, lng: 102.26708760598453, tier: 2, t
12   { name: 'Gerbang Keluar Depan UNIB', lat: -3.759153284887122, lng: 102.26691070286351, tier: 2, t
13   { name: 'Gerbang Masuk Belakang UNIB', lat: -3.7596380420268845, lng: 102.27516259098546, tier: 2
14   { name: 'Gerbang Keluar Belakang UNIB', lat: -3.7593149088938853, lng: 102.27622059999348, tier:
15   { name: 'Perpustakaan Pusat Universitas Bengkulu', lat: -3.7567420148766, lng: 102.27484309262306
16   { name: 'Gedung Serba Guna (GSG) UNIB', lat: -3.7575770654684173, lng: 102.2765918928578, tier: 2, type: 'f
17   { name: 'Gedung Olahraga (GOR) UNIB', lat: -3.7607612297254738, lng: 102.26758247749183, tier: 2, type: 'f
18   { name: 'Masjid Kampus Universitas Bengkulu', lat: -3.75897250873728, lng: 102.27593321314441, tier: 2, type: 'f
19   { name: 'Laboratorium Terpadu Universitas Bengkulu', lat: -3.758577679594874, lng: 102.27735320144158, tier:
20   { name: 'Gedung GB5 Universitas Bengkulu', lat: -3.7555147241605007, lng: 102.27644943719929, tier: 3, type:
21 ];

```

Gambar 3.2 Source Code Data Titik Lokasi

Source Code:

```

var manualPoints = [
  { name: 'Rektorat Universitas Bengkulu', lat: -3.758925991680951, lng:
102.27233254501347, tier: 1, type: 'rektorat' },
  { name: 'Fakultas Ekonomi dan Bisnis (FEB) UNIB', lat: -
3.761743030790814, lng: 102.26864780567482, tier: 1, type: 'fakultas' },
  { name: 'Fakultas Teknik Universitas Bengkulu', lat: -
3.7582743815464026, lng: 102.27738893738805, tier: 1, type: 'fakultas' },
  { name: 'Fakultas Ilmu Sosial dan Ilmu Politik (FISIP) UNIB', lat: -
3.759215588543014, lng: 102.27448693475885, tier: 1, type: 'fakultas' },
  { name: 'Fakultas Keguruan dan Ilmu Pendidikan (FKIP) UNIB', lat: -
3.7576116493071736, lng: 102.27506763393801, tier: 1, type: 'fakultas' },
  { name: 'Fakultas Pertanian Universitas Bengkulu', lat: -
3.757626592083991, lng: 102.27165981793573, tier: 1, type: 'fakultas' },
  { name: 'Fakultas Hukum Universitas Bengkulu', lat: -
3.7604697553707878, lng: 102.26844887116475, tier: 1, type: 'fakultas' },
  { name: 'Fakultas MIPA Universitas Bengkulu', lat: -3.756064554398636,
lng: 102.27476839254005, tier: 1, type: 'fakultas' },

```

```

    { name: 'Fakultas Kedokteran Universitas Bengkulu', lat: -
3.755049175852167, lng: 102.27799603213823, tier: 1, type: 'fakultas' },
    { name: 'Gerbang Masuk Depan UNIB', lat: -3.7599556451382616, lng:
102.26708760598453, tier: 2, type: 'gerbang' },
    { name: 'Gerbang Keluar Depan UNIB', lat: -3.759153284887122, lng:
102.26691070286351, tier: 2, type: 'gerbang' },
    { name: 'Gerbang Masuk Belakang UNIB', lat: -3.7596380420268845, lng:
102.27516259098546, tier: 2, type: 'gerbang' },
    { name: 'Gerbang Keluar Belakang UNIB', lat: -3.7593149088938853, lng:
102.27622059999348, tier: 2, type: 'gerbang' },
    { name: 'Perpustakaan Pusat Universitas Bengkulu', lat: -
3.7567420148766, lng: 102.27484309262306, tier: 2, type: 'fasilitas' },
    { name: 'Gedung Serba Guna (GSG) UNIB', lat: -3.7575770654684173,
lng: 102.2765918928578, tier: 2, type: 'fasilitas' },
    { name: 'Gedung Olahraga (GOR) UNIB', lat: -3.7607612297254738, lng:
102.26758247749183, tier: 2, type: 'fasilitas' },
    { name: 'Masjid Kampus Universitas Bengkulu', lat: -3.75897250873728,
lng: 102.27593321314441, tier: 3, type: 'masjid' },
    { name: 'Laboratorium Terpadu Universitas Bengkulu', lat: -
3.758577679594874, lng: 102.27735320144156, tier: 3, type: 'lab' },
    { name: 'Gedung GB5 Universitas Bengkulu', lat: -3.7555147241605007,
lng: 102.27644943719929, tier: 3, type: 'lab' }
];

```

Penjelasan Source Code:

Variabel `manualPoints` berfungsi sebagai basis data spasial statis (*static spatial database*) yang menyimpan representasi terstruktur dari seluruh titik-titik POI (*Point of Interest*) utama di kawasan kampus UNIB yang didefinisikan secara manual oleh pengembang.

Variabel `manualPoints` dideklarasikan sebagai sebuah *array* JavaScript yang berisi 19 objek literal. Setiap objek merepresentasikan satu titik lokasi dan memiliki lima properti yang terdefinisi dengan ketat. Properti `name` berisi string nama lokasi yang akan ditampilkan kepada pengguna. Properti `lat` dan

lng menyimpan koordinat geografis (*latitude* dan *longitude*) dalam format desimal dengan presisi tinggi (hingga 16 digit signifikan), yang merupakan nilai koordinat WGS84 (*World Geodetic System 1984*) — sistem referensi koordinat yang digunakan oleh GPS dan OpenStreetMap. Properti *tier* menyimpan nilai integer (1, 2, atau 3) yang menentukan prioritas visibilitas penanda pada sistem *Zoom Tiering* dinamis, di mana nilai 1 (mis. Rektorat, Fakultas) mengindikasikan lokasi paling penting yang tampil pada *zoom* paling rendah, nilai 2 (mis. Gerbang, Perpustakaan) tampil pada *zoom* menengah, dan nilai 3 (mis. Masjid, Laboratorium) baru tampil pada *zoom* tinggi. Properti *type* berisi string yang mengklasifikasikan kategori lokasi.

3.3.3 Sistem Zoom Tiering Dinamis

```
map.on('zoomend', () => {
  var zoom = map.getZoom();

  poiMarkers.forEach(item => {
    var isVisible = false;

    if (item.tier === 1 && zoom >= 14) isVisible = true;
    else if (item.tier === 2 && zoom >= 16) isVisible = true;
    else if (item.tier === 3 && zoom >= 17.5) isVisible = true;
    else if (item.tier === 4 && zoom >= 18.5) isVisible = true;

    if (isVisible && !map.hasLayer(item.marker)) {
      item.marker.addTo(map);
    } else if (!isVisible && map.hasLayer(item.marker)) {
      map.removeLayer(item.marker);
    }
  });
});
```

Gambar 3.3 Source Code Zoom Tiering

Source Code:

```
map.on('zoomend', () => {
  var zoom = map.getZoom();

  poiMarkers.forEach(item => {
    var isVisible = false;

    if (item.tier === 1 && zoom >= 14) isVisible = true;
    else if (item.tier === 2 && zoom >= 16) isVisible = true;
```

```

else if (item.tier === 3 && zoom >= 17.5) isVisible = true;
else if (item.tier === 4 && zoom >= 18.5) isVisible = true;

if (isVisible && !map.hasLayer(item.marker)) {
    item.marker.addTo(map);
} else if (!isVisible && map.hasLayer(item.marker)) {
    map.removeLayer(item.marker);
}
});
});

```

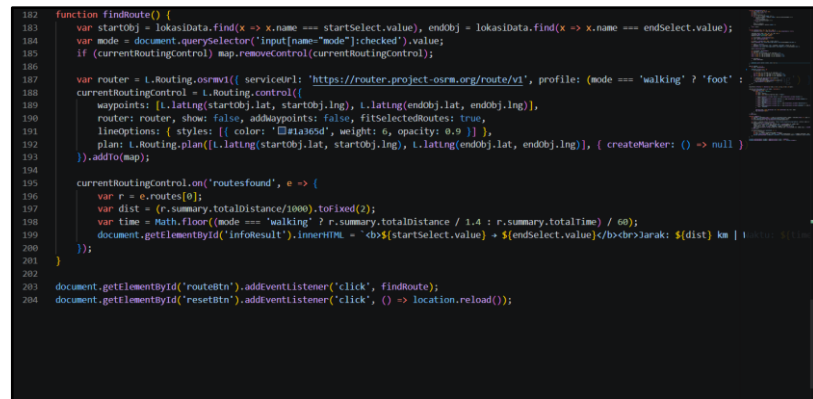
Penjelasan Source Code:

Segmen ini mengimplementasikan sistem *Zoom Tiering* dinamis sebuah mekanisme kontrol visibilitas adaptif yang secara otomatis memunculkan atau menyembunyikan penanda peta berdasarkan nilai *zoom* yang aktif, sehingga peta tidak penuh sesak oleh label pada saat *zoom* rendah.

Mekanisme ini bekerja dengan mendengarkan event '*zoomend*' yang dipancarkan oleh objek *map* setiap kali pengguna selesai melakukan operasi *zoom*. Saat event terpicu, nilai *zoom* saat ini diperoleh melalui *map.getZoom()* dan disimpan ke variabel lokal *zoom*.

Fungsi *callback* kemudian melakukan iterasi terhadap seluruh elemen dalam array *poiMarkers* menggunakan *forEach()*. Untuk setiap elemen *item*, variabel *isVisible* dihitung berdasarkan logika kondisional bertingkat yang mencocokkan properti *item.tier* dengan nilai ambang batas (*threshold*) *zoom* yang telah ditentukan: *tier 1* (lokasi utama) muncul saat *zoom* >= 14, *tier 2* (fasilitas penting) muncul saat *zoom* >= 16, *tier 3* (lokasi detail) muncul saat *zoom* >= 17.5, dan *tier 4* (lokasi KML tambahan) muncul saat *zoom* >= 18.5. Logika kondisional selanjutnya menerapkan prinsip efisiensi dengan memeriksa *map.hasLayer(item.marker)* sebelum melakukan operasi DOM: *item.marker.addTo(map)* hanya dipanggil jika *isVisible* adalah *true* DAN marker belum ada di peta, dan *map.removeLayer(item.marker)* hanya dipanggil jika *isVisible* adalah *false* DAN marker sedang ada di peta. Pola ini mencegah operasi yang tidak perlu dan menjaga performa aplikasi.

3.3.4 Fungsi Pencarian Rute Terpendek



Gambar 3.4 Source Code Pencarian Rute Terpendek

Source Code:

```
function findRoute() {  
    var startObj = lokasiData.find(x => x.name === startSelect.value), endObj = lokasiData.find(x => x.name === endSelect.value);  
    var mode = document.querySelector('input[name="mode"]:checked').value;  
    if (currentRoutingControl) map.removeControl(currentRoutingControl);  
  
    var router = L.Routing.osrmv1({ serviceUrl: 'https://router.project-osrm.org/route/v1', profile: (mode === 'walking' ? 'foot' : 'driving') });  
    currentRoutingControl = L.Routing.control({  
        waypoints: [L.latLng(startObj.lat, startObj.lng), L.latLng(endObj.lat, endObj.lng)],  
        router: router, show: false, addWaypoints: false, fitSelectedRoutes: true,  
        lineOptions: { styles: [{ color: '#1a365d', weight: 6, opacity: 0.9 }] },  
        plan: L.Routing.plan([L.latLng(startObj.lat, startObj.lng), L.latLng(endObj.lat, endObj.lng)], { createMarker: () => null })  
    }).addTo(map);  
  
    currentRoutingControl.on('routesfound', e => {  
        var r = e.routes[0];  
        var dist = (r.summary.totalDistance/1000).toFixed(2);  
        var time = Math.floor((mode === 'walking' ? r.summary.totalDistance / 1.4 : r.summary.totalTime) / 60);  
        document.getElementById('infoResult').innerHTML = `<b>${startSelect.value} - ${endSelect.value}</b><br>Jarak: ${dist} km | Waktu: ${time} menit`;  
    });  
    document.getElementById('routeBtn').addEventListener('click', findRoute);  
    document.getElementById('resetBtn').addEventListener('click', () => location.reload());  
}
```

```

        var time = Math.floor((mode === 'walking' ? r.summary.totalDistance /
1.4 : r.summary.totalTime) / 60);

        document.getElementById('infoResult').innerHTML
        =
        `<b>${startSelect.value} → ${endSelect.value}</b><br>Jarak: ${dist} km |
Waktu: ${time} menit`;

    });
}

```

```

document.getElementById('routeBtn').addEventListener('click', findRoute);
document.getElementById('resetBtn').addEventListener('click',    ()    =>
location.reload());

```

Penjelasan Source Code:

Fungsi `findRoute()` adalah inti komputasional dari seluruh aplikasi. Ia bertanggung jawab untuk mengeksekusi seluruh alur pencarian rute terpendek: dari pengambilan data input, komunikasi dengan server OSRM (yang secara internal menjalankan Multilevel Dijkstra dan Contraction Hierarchies), visualisasi rute di peta, hingga penghitungan dan tampilan output informasi jarak dan estimasi waktu.

Pada tahap pertama, fungsi menggunakan metode `Array.find()` untuk mencari objek data lengkap pada `lokasiData` yang namanya cocok dengan nilai yang dipilih pada `startSelect.value` dan `endSelect.value`. Nilai mode transportasi diperoleh dengan *selector* CSS `'input[name="mode"]:checked'` yang mengambil nilai elemen `<input type="radio">` yang sedang aktif.

Sebelum membuat rute baru, kondisi `if (currentRoutingControl)` memastikan bahwa objek kontrol rute yang sebelumnya aktif dihapus terlebih dahulu dari peta menggunakan `map.removeControl()`, mencegah penumpukan layer rute. Objek router OSRM dibuat dengan `L.Routing.osrmv1()` dengan menentukan `serviceUrl` ke server publik OSRM dan `profile` yang dipilih secara dinamis: string `'foot'` untuk mode `'walking'` atau `'driving'` untuk mode kendaraan.

Objek `L.Routing.control()` kemudian dikonfigurasi dan ditambahkan ke peta. Opsi `show: false` menyembunyikan panel instruksi belokan bawaan Leaflet Routing Machine karena sistem ini menggunakan `<div`

`id="infoResult">` sebagai panel output kustom. Opsi `fitSelectedRoutes: true` secara otomatis menyesuaikan `viewport` peta agar seluruh rute terlihat. Opsi `plan` dengan `createMarker: () => null` mencegah tampilnya penanda `waypoint` bawaan Leaflet Routing Machine yang akan tumpang tindih dengan penanda POI kustom.

Event listener `'routesfound'` menangkap respons asinkronus dari OSRM. Objek rute pertama `e.routes[0]` diambil dan data ringkasannya diekstrak: `r.summary.totalDistance` (dalam meter) dikonversi ke kilometer dengan pembagian 1000 dan diformat ke 2 desimal menggunakan `.toFixed(2)`, hasilnya disimpan ke `dist`. Estimasi waktu (**time**) dihitung secara berbeda tergantung mode: untuk mode `'walking'`, total jarak dibagi kecepatan jalan kaki rata-rata manusia sebesar 1.4 meter/detik (≈ 5 km/jam) karena data `r.summary.totalTime` dari OSRM untuk profil jalan kaki tidak selalu tersedia, sementara untuk mode kendaraan langsung menggunakan `r.summary.totalTime` (dalam detik). Hasil kalkulasi akhirnya dikonversi ke menit dengan `Math.floor(.../ 60)` dan dirender ke elemen `<div id="infoResult">`.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Tampilan Antarmuka Utama Aplikasi



Gambar 4.1 Tampilan Antarmuka Utama UNIB Route Navigation System

Penjelasan Antarmuka:

Gambar 4.1 menampilkan antarmuka utama (*user interface*) aplikasi UNIB Route Navigation System pada kondisi awal (*default*) saat halaman web pertama kali dimuat. Tata letak aplikasi didesain menggunakan arsitektur antarmuka modern yang terdiri dari *header* navigasi statis berwarna biru gelap di bagian atas untuk menampilkan identitas sistem, serta area kanvas peta interaktif berbasis Leaflet.js yang mengisi seluruh sisa dimensi layar (*fullscreen map*).

Pada sisi kiri antarmuka, terdapat sebuah panel kontrol *overlay* yang didesain menggunakan pendekatan efek visual *glassmorphism* (panel melayang dengan latar semi-transparan). Panel ini bertindak sebagai pusat interaksi utama yang memuat elemen formulir masukan pengguna, meliputi: elemen masukan *dropdown* untuk memilih Titik Asal dan Titik Tujuan yang datanya di-*parsing* secara dinamis, opsi *radio button* untuk penentuan parameter mode perjalanan (Kendaraan atau Jalan Kaki), tombol eksekusi utama Cari Rute Terpendek, serta tombol utilitas Reset Peta. Sementara itu, pada area peta, sistem langsung memfokuskan titik pandang (*setView*) pada koordinat sentral kawasan Universitas Bengkulu dan mengeksekusi logika *Zoom Tiering* untuk merender ikon penanda (*marker*) lokasi-lokasi berprioritas tinggi (Tier 1).

4.2 Pengujian Fitur Zoom Tiering

Salah satu tantangan teknis utama dalam pengembangan Sistem Informasi Geografis (GIS) dengan jumlah titik lokasi yang padat adalah terjadinya penumpukan ikon penanda (*marker clutter*). Kondisi ini tidak hanya menurunkan tingkat keterbacaan dan estetika peta, tetapi juga membebani kinerja komputasi rendering pada sisi klien (*client-side*). Untuk mengatasi permasalahan tersebut, sistem navigasi ini mengimplementasikan fitur *Zoom Tiering* atau yang secara akademis dikenal dengan istilah *Semantic Zooming*.

Mekanisme algoritma ini bekerja dengan mengikat *event listener* zoomend pada objek peta, yang secara konstan akan mengevaluasi visibilitas setiap titik lokasi berdasarkan korelasi antara tingkat perbesaran peta saat ini (*zoom level*) dengan variabel bobot prioritas (*tier*) yang disematkan pada struktur data masing-masing lokasi. Pengujian pada bagian ini ditujukan untuk mendemonstrasikan kelancaran transisi visual dan akurasi logika sistem dalam merender titik lokasi kampus dari level makro (Tier 1) hingga level geospasial mikro (Tier 4) seiring dengan interaksi *zoom in* yang dilakukan oleh pengguna.

4.2.1 Level Zoom Tier 1



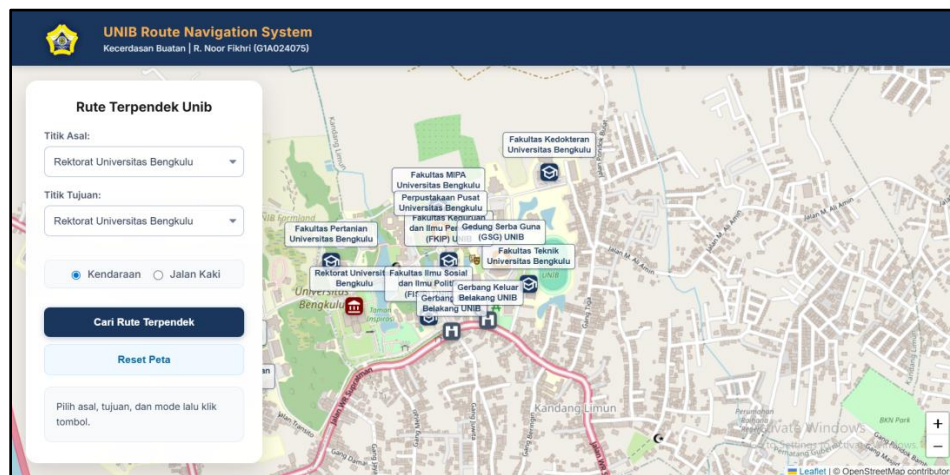
Gambar 4.2 Tampilan Peta pada Level Zoom Tier 1

Penjelasan:

Gambar 4.2 menampilkan kondisi peta pada level perbesaran makro atau *zoom out* maksimal (berada pada *zoom level* 14 hingga 15). Berdasarkan instruksi kondisional pada logika program, sistem hanya merender *marker* yang memiliki atribut prioritas tertinggi (Tier 1), yaitu fasilitas administratif

utama seperti Gedung Rektorat dan gedung-gedung dekanat fakultas (misalnya Fakultas Ekonomi dan Bisnis, Fakultas Hukum, dan Fakultas Kedokteran). Pendekatan ini secara efektif menjaga antarmuka peta tetap bersih (*clean UI*) dan membebaskan pandangan pengguna dari penumpukan ikon, sehingga letak geografis kawasan kampus secara garis besar dapat langsung dikenali sebelum pengguna melakukan penelusuran spasial yang lebih detail.

4.2.2 Level Zoom Tier 2

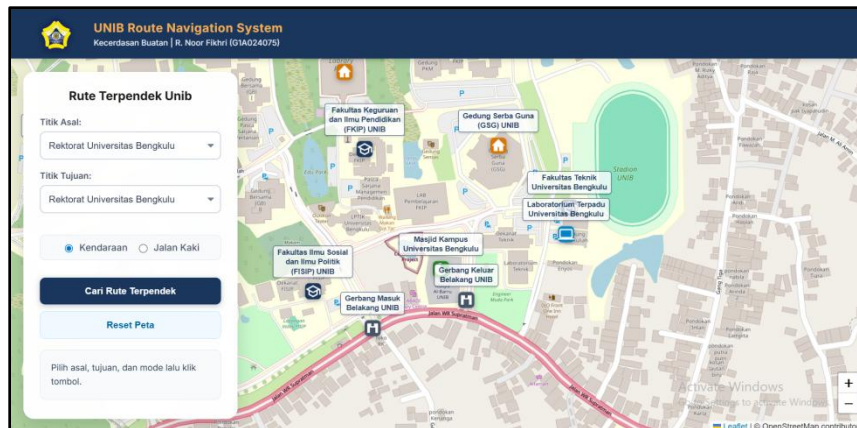


Gambar 4.3 Tampilan Peta pada Level Zoom Tier 1

Penjelasan:

Ketika pengguna melakukan interaksi perbesaran peta (*zoom in*) lebih dekat (memasuki *zoom level* 16), pendengar kejadian (*event listener*) pada sistem mendeteksi perubahan parameter tersebut dan secara dinamis memicu fungsi rendering untuk *marker* dengan status Tier 2. Seperti yang diilustrasikan pada Gambar 4.3, penanda untuk fasilitas umum berskala besar dan infrastruktur akses mulai dimunculkan. Hal ini terlihat dari munculnya ikon berwarna oranye yang merepresentasikan Gedung Serba Guna (GSG) dan Perpustakaan, serta ikon berwarna abu-abu yang menandakan titik Gerbang Masuk dan Gerbang Keluar kampus. Pada tahapan ini, *marker* berprioritas utama (Tier 1) tetap dipertahankan, sehingga sistem berhasil menyajikan penambahan detail informasi spasial secara progresif tanpa membuat pengguna kehilangan konteks orientasi wilayah utamanya.

4.2.3 Level Zoom Tier 3



Gambar 4.4 Tampilan Peta pada Level Zoom Tier 3

Penjelasan:

Pada tahap perbesaran peta yang lebih dalam (memasuki *zoom level* 17.5), sistem mengeksekusi logika rendering untuk titik lokasi berstatus Tier 3. Seperti yang diilustrasikan pada Gambar 4.4, pemunculan penanda mulai berfokus pada fasilitas menengah dan spesifik. Hal ini dibuktikan dengan munculnya ikon berwarna hijau yang merepresentasikan fasilitas ibadah (Masjid Kampus Universitas Bengkulu), serta ikon berwarna biru muda yang menandakan fasilitas akademik khusus seperti Laboratorium Terpadu Universitas Bengkulu. Pemunculan *marker* pada level ini dirancang secara khusus untuk mengakomodasi kebutuhan navigasi mikro pengguna yang mencari ruangan atau gedung spesifik di sekitar kawasan fakultas utama, melengkapi informasi struktural dari Tier 1 dan Tier 2 yang tetap divisualisasikan pada kanvas peta.

4.2.4 Level Zoom Tier 4



Gambar 4.5 Tampilan Peta pada Level Zoom Tier 4

Penjelasan:

Pada tingkat perbesaran maksimal (memasuki *zoom level* 18.5 ke atas), sistem merender seluruh titik lokasi secara komprehensif tanpa ada yang disembunyikan. Fase ini memicu pemunculan *marker* berstatus Tier 4, yang mayoritas datanya di-*parsing* secara dinamis dari file geospasial eksternal (*data_kampus.kml*). Seperti yang terlihat pada Gambar 4.5, informasi spasial yang disajikan mencapai tingkat granularitas tertinggi (mikro). Pengguna kini dapat melakukan navigasi presisi ke lokasi-lokasi spesifik di dalam suatu fakultas, seperti Program Studi Informatika, Sekretariat HIMASIF, hingga Laboratorium Energi Konversi Listrik. Keberhasilan sistem dalam merender detail pada Tier 4 ini sekaligus menyimpulkan keandalan algoritma *Semantic Zooming* yang dibangun; sistem mampu mengelola dan menyajikan puluhan hingga ratusan titik data geografis secara efisien tanpa mengorbankan performa *rendering* maupun estetika antarmuka peta pada level makro.

4.3 Pengujian Hasil Pencarian Route Terpendek

4.3.1 Route Mode Kendaraan



Gambar 4.6 Tampilan Hasil Pencarian Route Mode Kendaraan

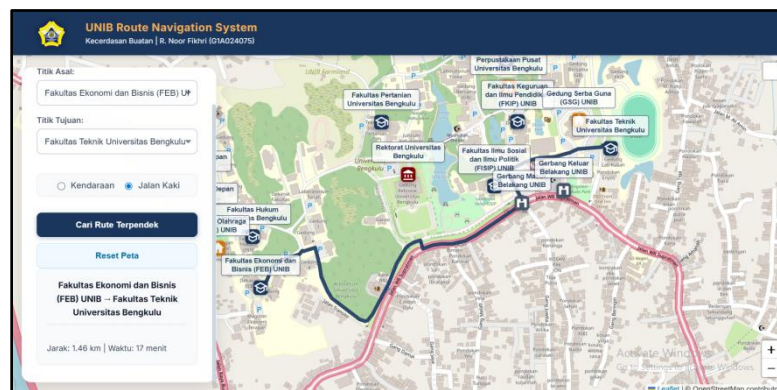
Penjelasan Output:

Gambar 4.6 mendemonstrasikan hasil pengujian fungsionalitas algoritma pencarian rute terpendek menggunakan parameter profil perjalanan kendaraan bermotor (*driving profile* pada OSRM). Pada skenario pengujian ini, pengguna menetapkan Titik Asal di "Fakultas Ekonomi dan Bisnis (FEB) UNIB" dan Titik Tujuan di "Fakultas Teknik Universitas Bengkulu".

Setelah perintah dieksekusi, sistem secara *real-time* merespons dengan menggambarkan sebuah *polyline* (garis lintasan tebal berwarna biru gelap) di

atas kanvas peta. Secara visual, garis rute tersebut tidak menarik garis lurus (Euclidean), melainkan secara akurat menelusuri jaringan jalan raya aktual yang sah untuk dilalui kendaraan di dalam kawasan kampus. Hal ini membuktikan bahwa algoritma (A* dan Dijkstra) berhasil melakukan komputasi pencarian jalur pada graf berbobot dari data spasial OpenStreetMap. Di sisi antarmuka, panel kontrol berhasil mengekstrak dan menampilkan hasil kalkulasi dari objek *response* API, yaitu rute optimal dengan total jarak tempuh sejauh 1.46 km dan estimasi waktu perjalanan selama 5 menit.

4.3.2 Rute Mode Jalan Kaki



Gambar 4.7 Tampilan Hasil Pencarian Rute Mode Jalan Kaki

Penjelasan Output:

Gambar 4.7 mengilustrasikan hasil komputasi pencarian rute dengan Titik Asal dan Titik Tujuan yang sama persis dengan pengujian sebelumnya, namun menggunakan parameter profil perjalanan pejalan kaki (*foot profile* pada OSRM). Pada mode ini, algoritma melakukan penyesuaian evaluasi bobot graf (*graph weighting*), di mana jalur pedestrian atau area yang tidak dapat dilalui kendaraan bermotor menjadi *node* yang valid untuk dieksplorasi.

Meskipun pada rute spesifik dari Fakultas Ekonomi dan Bisnis menuju Fakultas Teknik ini lintasan yang dihasilkan memiliki jarak tempuh yang identik dengan rute kendaraan (sejauh 1.46 km), terdapat perbedaan komputasi yang sangat signifikan pada hasil estimasi waktu. Sistem secara adaptif mengkalkulasi ulang waktu tempuh menjadi 17 menit. Perubahan metrik ini terjadi karena logika sistem secara otomatis mengubah dasar perhitungannya; mengabaikan batas kecepatan maksimal jalan raya, dan

membaginya dengan konstanta kecepatan berjalan manusia rata-rata (sekitar 1,4 meter/detik). Pengujian ini membuktikan fleksibilitas sistem navigasi dalam memberikan respons dan metrik *output* yang kontekstual sesuai dengan mode transportasi yang dipilih oleh pengguna.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi kode sumber, dan pengujian fungsional pada UNIB Route Navigation System, dapat disimpulkan bahwa sistem navigasi rute kampus Universitas Bengkulu berbasis Web GIS telah berhasil dibangun secara fungsional menggunakan teknologi *front-end* modern seperti HTML5, CSS3, JavaScript, dan pustaka pemetaan interaktif Leaflet.js. Integrasi Algoritma Dijkstra dan A* (A-Star) juga berhasil diimplementasikan secara optimal melalui pemanfaatan antarmuka pemrograman aplikasi (API) dari Open Source Routing Machine (OSRM). Algoritma tersebut terbukti mampu melakukan kalkulasi jalur terpendek secara *real-time* dan adaptif dalam menyesuaikan metrik jarak maupun estimasi waktu berdasarkan mode transportasi kendaraan dan jalan kaki. Selain itu, mekanisme *Zoom Tiering* dinamis (*Semantic Zooming*) berhasil dikembangkan untuk memecahkan masalah penumpukan ikon penanda (*marker clutter*) dengan mengatur visibilitas titik lokasi ke dalam empat tingkatan yang merespons perubahan perbesaran peta secara otomatis.

5.2 Saran

Meskipun sistem telah beroperasi dengan baik sesuai batasan masalah yang ditetapkan, terdapat beberapa saran yang dapat dipertimbangkan untuk pengembangan sistem lebih lanjut. Pada aspek arsitektur, disarankan untuk mengembangkan sisi *back-end* dan mengintegrasikan sistem manajemen basis data terpusat seperti MySQL untuk memudahkan pengelolaan titik lokasi kampus, mengingat saat ini sistem masih sepenuhnya beroperasi di sisi klien dengan pendefinisian data lokasi yang statis. Selanjutnya, pengembangan fitur deteksi lokasi perangkat pengguna (*geolocation*) secara otomatis sangat direkomendasikan agar pengguna tidak perlu memilih koordinat asal secara manual melalui menu prapilih. Dari sisi infrastruktur, karena operasional komputasi rute saat ini bergantung pada konektivitas internet menuju server OSRM publik, penggunaan server OSRM mandiri (*self-hosted*) yang ditempatkan pada peladen Universitas Bengkulu sangat disarankan untuk menekan tingkat latensi jaringan dan memungkinkan sistem diakses melalui mode intranet.

DAFTAR PUSTAKA

- Amin, A., & Hendrik, B. (2025). Analisis penerapan algoritma Dijkstra dalam optimasi penentuan rute: Sebuah kajian literatur sistematis. *Journal of Education Research*, 6(1), 100–106. <https://doi.org/10.37985/jer.v6i1.2155>
- Akbar, M. R., Zufria, I., & Harahap, A. M. (2024). Implementasi algoritma A Star pada sistem informasi geografis sekolah luar biasa di Kota Medan. *Journal of Computers and Digital Business*, 3(1), 18–25. <https://doi.org/10.56427/jcbd.v3i1.243>
- Suhendri, S., Abdurahman, D., & Maulana, D. I. (2021). Implementasi algoritma A-Star untuk pemetaan lokasi sarana kesehatan Kabupaten Majalengka berbasis Geographic Information System (GIS). *INFOTECH Journal*, 7(2), 57–65. <https://doi.org/10.31949/infotech.v7i2.1512>
- Umar, R., Yudhana, A., & Prayudi, A. (2021). Analisis perbandingan algoritma Dijkstra, A-Star, dan Floyd Warshall dalam pencarian rute terdekat pada objek wisata Kabupaten Dompu. *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIK)*, 8(2), 227–234. <https://doi.org/10.25126/jtik.0812866>
- Bhaskara, I. M. A., Kumara, I. M. S., Darma, I. G. W., Agus, I. K., & Raharja, W. (2024). Perbandingan algoritma Dijkstra dan Floyd-Warshall menggunakan Software Defined Network untuk rute terpendek. *Jurnal RESISTOR (Rekayasa Sistem Komputer)*, 7(1), 109–117. <https://ejournal.instiki.ac.id/index.php/jurnalresistor/article/view/1623>